

Bloco 4.4, Propostas: formulário de criação — status de entrega

Data: 2026-09-05. Branch: feature-dev-breno-automacao. Working tree, sem commit.

O que o bloco é

O formulário de criação de proposta, em seções, com react-hook-form + zod, no padrão de dois arquivos do repositório: `proposta-form.tsx` (client, com hooks) e `proposta-form-helpers.ts` (puro, importável de Server Component e Server Action). O molde é o `orcamento-form.tsx`.

Atividade 1: Seção Identificação

Concluída. Obra (select, obrigatório), número (obrigatório) e descrição.

O select de obra substitui o de cliente que Orçamentos tem, porque **proposta não tem `cliente_id`** — o cliente chega por obra. Por isso o rótulo de cada opção carrega o cliente entre parênteses, `OBRA-2026-01 – Edifício X (Construtora Y)`: sem isso, escolher a obra certa exigiria decorar códigos.

O número é `not null` e `unique (empresa_id, numero)` no banco, diferente do de orçamento, que é opcional. A unicidade não dá pra validar no form sem consultar o banco, então ela é tratada na volta do insert (atividade 8).

Atividade 2: Seção Datas

Concluída. `data_emissao` com default de hoje e `data_validade` opcional.

Uma validação foi além do banco: **validade anterior à emissão é recusada**. Não existe CHECK para isso, mas é erro de digitação garantido — a proposta nasceria vencida, já que `isPropostaVencida` compara `data_validade` com hoje.

Atividade 3: Seção Valores

Concluída. `valor_total`, `desconto` e `valor_final` calculado em runtime, read-only.

O campo de valor final é um `Input` desabilitado, alimentado por `calcularValorFinal(valor_total, desconto)`. É espelho, não fonte: quem calcula de verdade é o banco, onde `valor_final` é coluna `generated`.

Atividade 4: Seção Condições de pagamento

Concluída. Os quatro percentuais — `pct_sinal`, `pct_fd`, `pct_entrega_material`, `pct_medicao_instalacao` — em 0 a 100.

Atividade 5: validação da soma dos pct_*

Concluída, e é o ponto que exigiu mais cuidado.

O banco guarda **frações** (`numeric(5,4)`), **CHECK** `between 0 and 1`) e o **CHECK** `propostas_pct_soma` limita a soma a `1.0`. A UI trabalha em porcentagem. Se o form comparasse 0..100 contra o limite 1.0, **toda** proposta seria recusada — por isso existe `validarSomaPctForm`, que converte antes de validar.

O aviso é ao vivo: um bloco abaixo dos campos mostra a soma e, quando passa de 100%, fica vermelho antes de qualquer tentativa de salvar. Quando a soma é menor que 100%, mostra quanto restou — soma parcial **é válida** (o **CHECK** é `<=`, não `=`), e o resto é justamente o que o campo livre descreve.

Atividade 6: condicoes_pagamento e observacao

Concluída. Duas textareas, `condicoes_pagamento` na seção de pagamento (logo abaixo do indicador de soma, que é o que dá sentido a ela) e `observacao` em seção própria.

Atividade 7: zod

Concluída. `obra_id` obrigatório (`uuid`), `numero` obrigatório (máx. 50) e `desconto <= valor_total`, espelhando o **CHECK** `propostas_desconto_valido`.

As regras não estão escritas dentro do schema. `validarDesconto`, `calcularValorFinal` e `validarSomaPctForm` vivem em `src/lib/propostas.ts`, e o `superRefine` as chama. O motivo é testabilidade: `node --test` não resolve o alias `@/`, então arquivo dentro de `src/app/` não é testável na prática. Com as regras no `lib`, elas têm cobertura direta.

Detalhe que os testes travam: `validarDesconto` arredonda em duas casas antes de comparar. Sem isso, `0.1 + 0.2 = 0.30000000000000004` recusaria um desconto que o banco — `numeric(14,2)` — aceitaria.

Atividade 8: valor_final não entra no payload

Concluída. `formValuesToPayload` monta `valor_total` e `desconto` e nunca `valor_final`. Mandar a coluna `generated` faria o Postgres recusar o insert inteiro.

Na mesma linha, `pctFormToPayload` converte os quatro percentuais de volta pra fração antes de enviar.

Atividade 9: toast e redirect

Concluída. Sucesso mostra "Proposta criada com sucesso" e navega pra `/propostas/[id]`, mais `router.refresh()`.

No erro, o toast mostra mensagem traduzida, não o texto cru do Postgres. `mensagemDeErroProposta` cobre as seis violações possíveis; a mais provável é o número repetido, que sem tradução chegaria como `duplicate key value violates unique constraint propostas_empresa_id_numero_key`.

Feito além do checklist: a proposta nasce rascunho

O card tem nove atividades, todas acima. Isto aqui não está no checklist, mas sem isso o formulário não teria o que gravar em `status`.

`createProposta` fixa `status: 'rascunho'` — o form não tem campo de status, e mudança de status é o bloco 4.6.

A action repete a checagem de perfil por conta própria (admin ou comercial), e a rota `/propostas/nova` tem o mesmo guard: o `layout.tsx` de propostas libera `visualizador`, e guard de layout protege rota, não ação.

Verificação

Rodada em 2026-09-05, depois das correções de `docs/tecnicos/correcoes-listagens-e-obras.md` — são os números do plano completo no fim do dia, não de um instante intermediário.

CAMADA	RESULTADO
estático	<code>tsc --noEmit</code> limpo, lint sem warnings
unitário	46 casos, <code>fail 0</code>
build	31 rotas, iguais à baseline
runtime	25/25 rotas, 4 delas como visualizador
dados	11/11 checagens sob RLS em gc-dev
escrita	28/28 passos exercitando as Server Actions, com limpeza

Entre as rotas, `/propostas/nova` devolve 200 com as cinco seções do formulário como admin, e **307 como visualizador** — o guard da página, já que o layout de `/propostas` libera esse perfil.

A camada de escrita cobre este bloco em cheio: `createProposta` roda contra gc-dev, a proposta nasce `rascunho`, `valor_final` volta 900 (a coluna generated calculando $1000 - 100$), os `pct_*` chegam como fração, e o número repetido é recusado com a mensagem traduzida. A criação como `visualizador` é recusada por permissão.

Não validado: o clique. O formulário não foi preenchido em navegador, então a validação do zod no cliente, o estado do react-hook-form, o toast e a navegação pós-sucesso seguem sem exercício. O que roda é a action que o formulário chamaria.

Pendências

1. ~~Criação não exercitada pela interface~~. **Resolvida em 2026-09-05** pela camada de escrita (`scripts/validar-escrita.mjs`), que chama `createProposta` por HTTP contra gc-dev e apaga a proposta no fim. Sobra o clique — abrir o formulário em navegador e submeter uma vez, para exercitar zod no cliente, toast e navegação.
2. **Unicidade do número só falha no servidor**. É **decisão**, não pendência: o form não consulta o banco antes de submeter, e a camada de escrita confirma que a recusa chega como "Já existe uma proposta com esse número". Se incomodar, um `onBlur` consultando `numero` resolve.

Arquivos

ARQUIVO	O QUE É
<code>src/app/(app)/propostas/proposta-form-helpers.ts</code>	schema zod, <code>formValuesToPayload</code> , <code>propostaToFormValues</code>
<code>src/app/(app)/propostas/proposta-form.tsx</code>	o form em cinco seções, compartilhado com a edição
<code>src/app/(app)/propostas/nova/</code>	<code>page.tsx</code> , <code>nova-form.tsx</code> e <code>actions.ts</code> — a rota de criação e o <code>createProposta</code>
<code>src/lib/propostas.ts</code>	<code>validarDesconto</code> , <code>calcularValorFinal</code> , conversão de pct, <code>mensagemDeErroProposta</code>
<code>src/lib/propostas.test.ts</code>	casos das regras acima